WEBSTER 2026

TEAM: <SYNTAXRUSH>

TEAM ID

233

TEAM INFO

Member 1: Harsh Choudhary(20244071,ECE ,syntaxbuilder2006_76484)

Member 2: Mohit Bagri(20244107,ECE,james107f)

Member 3: Shiwansh Dwivedi(20244152,ECE,shero0123)

DESCRIPTION OF THE PROBLEM

In modern cloud architecture and automation systems, developers and automated processes frequently perform repetitive, multi-step computational tasks that depend on one another. Executing these workflows manually or via rigid, hardcoded scripts leads to significant operational bottlenecks:

- **Repetitive Computational Loops ("Temporal Loops"):** Developers are trapped in manual execution cycles for routine pipeline tasks, scheduled runs, and asynchronous events.
- **Lack of Visual & Deterministic Orchestration:** Hardcoded execution flows lack flexibility, making it difficult to visualize task dependencies, dynamic branching, or prevent circular execution loops (infinite cycles) before runtime.
- **Brittle Failure Handling:** Standard API calls and tasks often fail due to transient network glitches, rate limits, or downstream outages. Without proper retry policies, dead-letter queues, and deterministic ordering, a single failure crashes the entire pipeline without recovery or state tracking.
- **Zero Visibility:** Operating complex asynchronous jobs without real-time telemetry leaves engineers blind to where and why a workflow stalled or failed.

Website name: Kahn Engine

PROCESS FLOW OF THE WEBSITE

1. **Workflow Design (Canvas UI):** The user visually arranges nodes (Webhook, HTTP Request, Transform, Delay) and connects edges on the React Flow interface.
2. **Cycle Detection (Validation):** Before sending data to the server, the client runs a graph algorithm (e.g., Kahn's Algorithm) to ensure the graph contains no cyclic loops.
3. **Database Persistence:** Valid DAG layouts are saved in PostgreSQL as JSON definitions containing node configurations and directional edge connections.
4. **Trigger Ingestion:** An incoming external HTTP POST request hits a dynamic /api/v1/webhooks/:id route, or a node-cron daemon fires based on a scheduled cron expression.

5. **Job Enqueueing:** The server creates a unique execution_id, sets the run status to PENDING, and pushes the workflow payload into a Redis task queue.
6. **Topological Order Engine:** A decoupled worker process pops the job from Redis, retrieves the graph definition, and evaluates the topological order of execution (identifying parent-child relationships).
7. **Execution & Telemetry Loop:**
    - Nodes execute strictly sequentially or in parallel according to parent-child dependency mapping.
    - Execution logs, node statuses (RUNNING, SUCCESS, FAILED), and outputs are emitted in real-time over Server-Sent Events (SSE) to the frontend UI.
    - If a node throws an error, the worker executes configurable exponential backoff retries with jitter. If retries exhaust, the execution stalls, routes to a Dead-Letter Queue (DLQ), and flags the run as FAILED.
8. **Finalization:** Once all leaf nodes resolve successfully, the worker flags the entire run as SUCCESS and commits all telemetry payloads and completion timestamps to PostgreSQL

Features announcement

- **Visual Flow Builder:**
    - Drag-and-drop node graph builder canvas.
    - Support for core node types: **Triggers**, **Transforms**, **HTTP Requests**, **Webhooks**, and **Delays**.
    - Built-in **cycle-detection validation** to prevent users from creating circular connections.
- **Deterministic Execution Engine:**
    - Backend execution engine utilizing **topological sort**.
    - Guarantees that dependent child nodes execute strictly after upstream parent nodes successfully finish.
- **Execution History & Telemetry:**
    - Tracking for every run with per-step logs.
    - Capture of node execution payloads, timestamps, and live visual execution statuses (RUNNING, SUCCESS, FAILED).
- **Webhook & Cron Triggers:**
    - Ability to trigger workflows via external HTTP webhook endpoints.
    - Scheduled execution support using standard cron expressions with payload injection.

External libraries/APIs

1. **Frontend Workflow Editor:** @xyflow/react (React Flow) for native React component nodes and built-in edge routing.

2. **Frontend Data Management:** @tanstack/react-query for automatic API caching, polling, and real-time state synchronization.

3. **Backend Framework:** Fastify (Node.js) for high-throughput HTTP request processing and native Async/Await execution.

4. **Graph Logic:** graphlib and custom DFS routines for client and server DAG topological sorting and cycle detection.
5. **Distributed Queue:** BullMQ for Redis-backed background worker processing, concurrency control, and retries.
6. **Primary Database:** PostgreSQL with Prisma ORM for relational schemas and JSONB dynamic node storage.
7. **In-Memory Store:** Redis for BullMQ queue state, pub/sub messaging, and worker heartbeat management.
8. **Live Streaming:** Server-Sent Events (SSE) for unidirectional, real-time log streaming to the browser.
9. **Scheduled Triggers:** node-cron for parsing cron expressions and injecting scheduled tasks into the queue.

Core Engineering Architecture

1. Infrastructure Layout & Data Flow

- **Visual Canvas & API Gateway:** Fastify API Gateway receives canvas layouts and incoming triggers (Webhooks/Cron).
- **In-Memory Graph Validation:** Cycle detection runs in-memory before persisting graph definitions to PostgreSQL.
- **Decoupled Task Enqueueing:** Validated execution requests are immediately pushed to a Redis-backed BullMQ queue as job payloads.
- **Distributed Worker Processing:** Decoupled background workers consume jobs from Redis, compute topological node ordering, and run tasks asynchronously.
- **Live Telemetry Streaming:** Execution logs and node status changes bypass database read-polling by streaming directly from workers to clients via Server-Sent Events (SSE).

2. High Concurrency & Resource Load Plan

- **Non-Blocking Ingestion:** The API Gateway validates payloads and acknowledges incoming triggers in under 10ms, offloading execution to Redis.
- **Backpressure Control:** BullMQ workers operate with fixed concurrency caps, buffering heavy request spikes safely inside Redis without crashing downstream services.
- **Database Connection Management:** PostgreSQL connection pools are capped via Prisma ORM to prevent connection exhaustion during concurrent worker runs.
- **Worker Heartbeat Monitoring:** Workers send periodic heartbeats to Redis; if a worker fails mid-execution, stalled jobs are automatically reassigned to healthy nodes.

3. Engineering Trade-offs & Production Defenses

Graph Validation & Execution Ordering

- **The Naive Way:** Recursive graph traversals on live network requests, causing stack overflows and $O(N^2)$ execution delays on complex workflows.

- **Smart Engineering:** Kahn's Algorithm (Topological Sort) running double-pass validation (client-side pre-check + server-side in-memory array mapping).
- **Why It Wins:** Guarantees deterministic $O(V + E)$ execution order and rejects circular dependencies at the gateway before wasting worker compute resources.

Task Execution & Failure Recovery

- **The Naive Way:** Running multi-step tasks inside standard async/await blocks on the main web thread, failing instantly without retries or state recovery on API timeouts.
- **Smart Engineering:** Decoupled BullMQ worker queueing with configurable exponential backoff, jitter, and Dead-Letter Queue (DLQ) isolation.
- **Why It Wins:** Isolates task failures, prevents third-party API thundering herd issues, and routes unrecoverable failures to DLQ without crashing the pipeline.

Real-Time Telemetry & Log Streaming

- **The Naive Way:** Client HTTP short-polling (setInterval), flooding the primary database with repetitive read queries under high user loads.
- **Smart Engineering:** Unidirectional Server-Sent Events (SSE) streaming backed by Redis Pub/Sub.
- **Why It Wins:** Eliminates database read queries for active runs, reduces network header overhead, and maintains low latency under high concurrency.
-

Class diagram

